

Global Flight Network Dashboard, Project Report

Course: Graph Databases (KIU, Semester 8) · **Author:** Davit Maisuradze
Stack: Neo4j 5 (graph database) + NeoDash 2.4 (dashboard) + Docker + APOC

Contents

- 1. [Overview](#)
- 2. [The data and where it comes from](#)
- 3. [The graph model](#)
- 4. [Conversion and import pipeline](#)
- 5. [The dashboard, page by page](#)
- 6. [Key Cypher queries](#)
- 7. [Design decisions and challenges](#)
- 8. [How to run it](#)

1. Overview

ok so the basic idea of this project is to take the whole world's flight network, load it into **Neo4j**, and then build a **NeoDash** dashboard on top so you can actually click around and explore it. once you model it as a graph (airports are nodes, the routes between them are relationships) a lot of the interesting questions basically answer themselves: which airports are the biggest hubs, how do you get from country A to country B when there's no direct flight, can you fly a whole trip on a single airline, stuff like that.

the dashboard ended up with **four pages and 47 reports**, and two of those pages are fully interactive (you pick things from dropdowns and everything updates). the part i'm most happy with is the **automatic route building**: you pick any two countries (or two airports) and it figures out the shortest connecting itinerary through intermediate airports, even when there is no direct flight at all. for example there's nothing direct from **Georgia → Japan**, but the dashboard builds it as **TBS → URC → PEK → KIX**.

2. The data and where it comes from

i used the **OpenFlights** dataset (<https://github.com/jpatokal/openflights>, it's under the Open Database License). it's a pretty well-known open dataset of global aviation data that people maintain over time. i only needed five of its raw files:

File	Rows	What it contains
airports.dat	7,698	Airport id, name, city, country, IATA/ICAO codes, latitude/longitude , altitude, timezone
airlines.dat	6,162	Airline id, name, IATA/ICAO codes, callsign, country, active flag
routes.dat	67,663	Operating airline + source airport + destination airport + stops + aircraft equipment
countries.dat	261	Country name and ISO code
planes.dat	246	Aircraft type name + IATA/ICAO code

i went with flight data because it's already a graph in real life: airports are the nodes, routes are the edges. so it shows off the graph stuff (hubs, connectivity, shortest paths) way better than a normal relational table would, and on top of that it's easy for anyone to get, you don't need to explain what an airport is.

3. The graph model

Node types (5):

- **Airport**: airportId, name, city, country, iata, icao, latitude, longitude, altitude (meters), location (spatial point)
- **Airline**: airlineId, name, iata, icao, callsign, country, active
- **Country**: name, isoCode, dafifCode
- **City**: key, name, country
- **Plane**: iata, icao, name (aircraft-type lookup, joined to routes by equipment code)

Relationship types (5):

- `(:Airport)-[:LOCATED_IN]->(:Country)`
- `(:Airport)-[:IN_CITY]->(:City)`
- `(:City)-[:IN_COUNTRY]->(:Country)`
- `(:Airline)-[:REGISTERED_IN]->(:Country)`
- `(:Airport)-[:ROUTE {airline, airlineId, stops, equipment, distanceKm}]->(:Airport)` is the important one. it's the relationship that carries the actual properties: which airline flies it, and the **geographic distance** of the route. one thing to keep in mind is there can be **several ROUTE edges between the same two airports**, one per airline that flies it.

4. Conversion and import pipeline

the raw OpenFlights files are headerless csv and they use `\N` for nulls, so i didn't have to edit them by hand at all, Neo4j just reads them straight with `LOAD CSV`.

1. `scripts/download_data.sh` grabs the `.dat` files into `./data`, which is mounted into the Neo4j container's import folder.
2. `docker-compose.yml` starts Neo4j 5.26 (with the **APOC** plugin) and NeoDash.
3. `scripts/import.sh` runs five Cypher scripts (`import/01...05`) in order: constraints and indexes first, then countries, airports (+cities), airlines, and routes last.

a couple of things i had to handle in basically every load script:

- **Nulls**. in OpenFlights a null is the literal text `\N`, not an empty field, so i guard every column with `CASE WHEN row[x] = '\N' THEN null ELSE ... END`.
- **Batching**. the big loads (airports, routes) use `CALL { ... } IN TRANSACTIONS` so it commits in chunks instead of trying to do everything in one giant transaction.
- **Meters, not feet**. OpenFlights gives altitude in feet, which annoyed me, so i convert it to meters right as it loads (`round(toFloat(row[8]) * 0.3048)`). every altitude in the project is in meters.

the part i think is actually the most interesting: OpenFlights has **no distance column at all** (and no price either), so i just **compute the great-circle distance for every route myself in Cypher, at import time**. each airport gets a spatial `point` from its lat/long, and then each route stores

```
distanceKm = round(point.distance(src.location, dst.location) / 1000)
```

this one computed property is what makes everything else possible, the distance histogram, all the route/leg tables, and especially the shortest-path routing. the catch is the airports have to be loaded **before** the routes, otherwise there's no `location` to measure from, that's the whole reason the import order (01 → 05) is fixed.

5. The dashboard, page by page

the dashboard is split into four pages by topic. tables can be downloaded as **CSV** and the charts/maps can be exported as **PNG**.

Page 1: Airport Network Analysis (overview)

just a big-picture look at the whole network.

- **KPI cards**: total airports, routes, airlines, countries.
- **Bar chart**: top 15 busiest airports by total routes (in + out).
- **Graph**: how the **10 largest hubs** connect to each other, as a node-link diagram.
- **Line chart**: how route distances are distributed, in 500 km buckets.
- **Pie chart**: top 10 airlines by number of routes.
- **Table** (CSV): the full airport-connectivity ranking.
- **Maps**: the major airports plotted from their coordinates, plus a global airport **density heatmap**.

Page 2: Country & Route Connectivity (interactive)

you pick a **source** and a **destination** country and everything below reacts to it.

- Two **dropdown selectors** (`$neodash_source_country` , `$neodash_destination_country`).
- **KPIs**: how many direct routes there are between the two countries, and the shortest **connecting** distance.
- **Bar charts**: top destination countries from the source, and top countries by airport count.
- **Table** (CSV): every direct route between the two countries.
- **Connecting-flight map**: this is the main thing, the **auto-built shortest itinerary** between the two countries' main hubs, drawn on a real map with **numbered stops** (`1 · TBS → 2 · URC → ...`). it works even when there's no direct flight.
- **Leg-by-leg table** (CSV): each hop of that itinerary, so the airports, the airline, and the distance.
- **Map**: the airports of both selected countries.

Page 3: Aviation Records & Trivia (extremes)

basically the fun "world records" of the network.

- **Pie chart**: most-used aircraft types (joined to the `Plane` nodes; the Airbus A320 wins).
- **Tables** (CSV): the longest non-stop routes (SYD↔DFW, 13,824 km), the highest-altitude airports (Daocheng Yading, 4,411 m), the most isolated airports, and the shortest scheduled flight in the world (**Papa Westray ↔ Westray, 3 km**).
- **Bar chart**: busiest city-to-city connections.

Page 4: Airport & Airline Explorer (interactive trip planner)

you pick a **From** airport, a **To** airport, and an **airline**. the page is built in three layers so all three selectors actually work **together**:

1. **From the origin**: a map + table of everywhere you can fly **non-stop on the chosen airline**, plus a reachability line chart showing how many airports you can reach within 1–4 stops (the **small-world effect**).
2. **From → To, any airline**: a yes/no "can i fly direct?" card, plus the **best connecting itinerary** (shortest distance, numbered map + leg table) that's allowed to mix airlines.
3. **From → To on one airline**: the main feature here, a verdict card ("**Emirates: CDG → JFK in 2 leg(s)**") plus a numbered map and leg table for the fewest-stop trip you can do **entirely on the selected airline**.

the chosen airline's full route network (graph + map) is at the bottom just for context.

6. Key Cypher queries

these are the queries that actually do the work, the ones worth looking at.

6.1 Great-circle distance at import (`import/05_routes.cypher`)

turns two sets of coordinates into the weight that every later query depends on:

```
MATCH (src:Airport {airportId: row.srcId}), (dst:Airport {airportId: row.dstId})
CREATE (src)-[:ROUTE {
  airline: row.airline,
  distanceKm: round(point.distance(src.location, dst.location) / 1000)
}]>(dst)
```

6.2 Busiest hubs by degree (Page 1)

counts incoming + outgoing routes per airport with the `COUNT { }` subquery instead of a join:

```
MATCH (a:Airport) WHERE a.iata IS NOT NULL

WITH a, COUNT { (a)-[:ROUTE]>() } + COUNT { (a)<-[:ROUTE]-() } AS routes

RETURN a.iata, routes ORDER BY routes DESC LIMIT 15
```

6.3 Country-to-country connecting route, *the main one* (Page 2)

there's no direct Georgia → Japan flight so the dashboard has to build one. it grabs the **top 3 hubs of each country**, runs **Dijkstra weighted by distanceKm** between every source/destination pair, and keeps the cheapest path. i use three hubs per side instead of one because with just one it sometimes found nothing or a bad path, three makes it work for almost any country pair:

```
MATCH (s:Airport)-[:LOCATED_IN]>(:Country {name:$neodash_source_country})
WITH s, COUNT { (s)-[:ROUTE]>() } AS sd WHERE sd > 0
WITH s ORDER BY sd DESC LIMIT 3
WITH collect(s) AS sources

MATCH (d:Airport)-[:LOCATED_IN]>(:Country {name:$neodash_destination_country})
WITH sources, d, COUNT { (d)<-[:ROUTE]-() } AS dd WHERE dd > 0
WITH sources, d ORDER BY dd DESC LIMIT 3
WITH sources, collect(d) AS dests
UNWIND sources AS src
UNWIND dests AS dst
CALL apoc.algo.dijkstra(src, dst, 'ROUTE>', 'distanceKm') YIELD path, weight
RETURN path ORDER BY weight ASC LIMIT 1
```

6.4 Numbered stops on the map via virtual nodes (Page 2 & 4)

NeoDash maps can't draw arrowheads, which was annoying, so instead i show direction by **numbering the stops**. i rebuild the path with **virtual nodes** (`apoc.create.vNode`) where the label carries the stop number, and **virtual relationships** carrying the leg order. these only exist for the map, they don't touch the real stored graph:

```
WITH nodes(path) AS ns, relationships(path) AS rs
WITH [i IN range(0, size(ns)-1) | apoc.create.vNode(['Airport'], {
  iata: toString(i+1) + ' · ' + ns[i].iata,
  latitude: ns[i].latitude, longitude: ns[i].longitude, location: ns[i].location})] AS vn, rs
UNWIND range(0, size(vn)-2) AS j
RETURN vn[j] AS a,
apoc.create.vRelationship(vn[j], 'LEG', {seq: j+1, distanceKm: rs[j].distanceKm}, vn[j+1]) AS r,
vn[j+1] AS b
```

6.5 Single-airline routing (Page 4)

this answers "can i fly the whole trip on one airline?". it's `shortestPath` with the airline constraint pushed into the search as an `all(...)` predicate, so it finds the fewest-stop path and stops early (it stays sub-second even for huge airlines):

```
MATCH (a:Airline {name:$neodash_airline}) WITH a.l.iata AS code LIMIT 1
MATCH (from:Airport {iata:$neodash_from_airport}), (to:Airport {iata:$neodash_to_airport})
MATCH p = shortestPath( (from)-[:ROUTE* 1..4]->(to) )
WHERE all(r IN relationships(p) WHERE r.airline = code)
RETURN p
```

6.6 Reachability / small-world effect (Page 4)

`apoc.neighbors.byhop` gives the newly-reached airports at each hop, and a running sum turns that into a cumulative reachability curve:

```
MATCH (a:Airport {iata:$neodash_from_airport})
CALL apoc.neighbors.byhop(a, 'ROUTE>', 4) YIELD nodes
WITH collect(size(nodes)) AS perHop
UNWIND range(0, size(perHop)-1) AS i
RETURN (i+1) AS Stops, reduce(s=0, j IN range(0,i) | s + perHop[j]) AS ReachableAirports
ORDER BY Stops
```

7. Design decisions and challenges

- **Computing the distance instead of looking it up.** the dataset just doesn't have distances, so deriving `distanceKm` from the spatial points (§4) was the thing that unlocked everything else, it's the weight behind every Dijkstra and every distance ranking.
- **Three hubs per country, not one.** my first version only routed between the single busiest airport of each country, and it sometimes gave back nothing or a clearly worse path. once i searched the top three hubs on each side i got shorter, more reliable routes (Georgia → Japan went from a 5-hop, 7,986 km path down to a 3-hop, 7,642 km path).
- **shortestPath instead of weighted variable-length for the single-airline route.** the naive `(:ROUTE*1..4 {airline})` pattern took **9–13 seconds** for big carriers like Ryanair, way too slow for a dashboard. rewriting it as `shortestPath` with the airline as an `all()` predicate lets Neo4j push the filter into a breadth-first search that bails out early, and it drops to **sub-second**.
- **Maps don't have arrows.** NeoDash just draws plain lines between airports, so i show direction by numbering the stops with the virtual nodes from §6.4.
- **Tables vs charts.** a few reports started as bar/pie charts where the category/value mapping only had one setup that made sense and broke if you switched it, so i turned those into tables (no such mapping to break) or into pies with the mapping fixed in the JSON.

8. How to run it

```
bash scripts/download_data.sh # fetch OpenFlights data into ./data

docker compose up -d # start Neo4j (+APOC) and NeoDash

bash scripts/import.sh # load the graph, then print node/edge counts
```

then open NeoDash at <http://localhost:5005>, connect to `neo4j://localhost:7687` (user `neo4j`, password `flights123`), and **Load** `dashboard/flights_dashboard.json`.

to wipe everything and start over (including the database volume):

```
docker compose down -v
```